

Ciência da Computação
Compiladores

Trabalho 01

Pascalino: um analisador léxico e IDE para um dialeto da linguagem Pascal

Relatório técnico

Aluno(s):FABRÍCIO LOPES LIMA DO AMARAL, LUCAS BORGES DO CARMO,
RAPHAEL FIGUEIREDO SECCHIN

Professor: Rodrigo Freitas Silva

1. Introdução

Este relatório apresenta o analisador léxico desenvolvido para o Pascalino, um dialeto da linguagem Pascal, e a IDE em Java/Swing que o acompanha. A ferramenta lê o código-fonte, ignora espaços e comentários, gera a sequência de tokens e informa os erros léxicos encontrados.

2. A linguagem Pascalino

2.1 Características léxicas

A definição léxica do Pascalino segue as seguintes regras:

- A linguagem não diferencia letras maiúsculas de minúsculas. Para consultar as palavras reservadas, o lexema é convertido para maiúsculas; já os identificadores são armazenados em minúsculas.
- Os identificadores podem ter até 15 caracteres. Eles devem começar com uma letra e, depois disso, podem conter letras, dígitos e sublinhado.
- As palavras reservadas não podem ser usadas como identificadores. Além das palavras listadas no Anexo I, o Pascalino inclui FOR, TO, DOWNT, TYPE e RECORD como extensões.
- Os números podem ser inteiros ou reais, sempre no formato dígitos.dígitos. Os literais são delimitados por apóstrofos e não podem ocupar mais de uma linha.
- Os comentários podem começar com //, quando ocupam o restante da linha, ou ser delimitados por { } e (* *). Assim como os espaços em branco, eles são ignorados e não geram tokens.

A gramática de referência está apresentada no Anexo I. Para este trabalho, ela foi ampliada com o laço for e com os tipos enumeração e registro. Nesta etapa, foram implementados os tokens correspondentes e as verificações descritas na seção 5.5. As produções previstas para a análise sintática são:

```
declTipo      -> ID = ( ID conjuntoIds ) ;           (tipo enumeração)
               | ID = RECORD declVarList END ;       (tipo registro)
inst          -> ... | FOR ID := expr TO expr DO inst
               | FOR ID := expr DOWNT expr DO inst
variavel      -> ID | ID [ exprOp ] | ID . ID        (acesso a campo de registro)
```

3. Arquitetura do programa

A organização do programa combina o padrão Model-View-Presenter, usado na interface, com o padrão Strategy, usado na geração de avisos. Dessa forma, a interface não precisa conhecer os detalhes do analisador: o PresenterAnálise atua como intermediário entre os dois. A Figura 1 apresenta os módulos que compõem a solução.

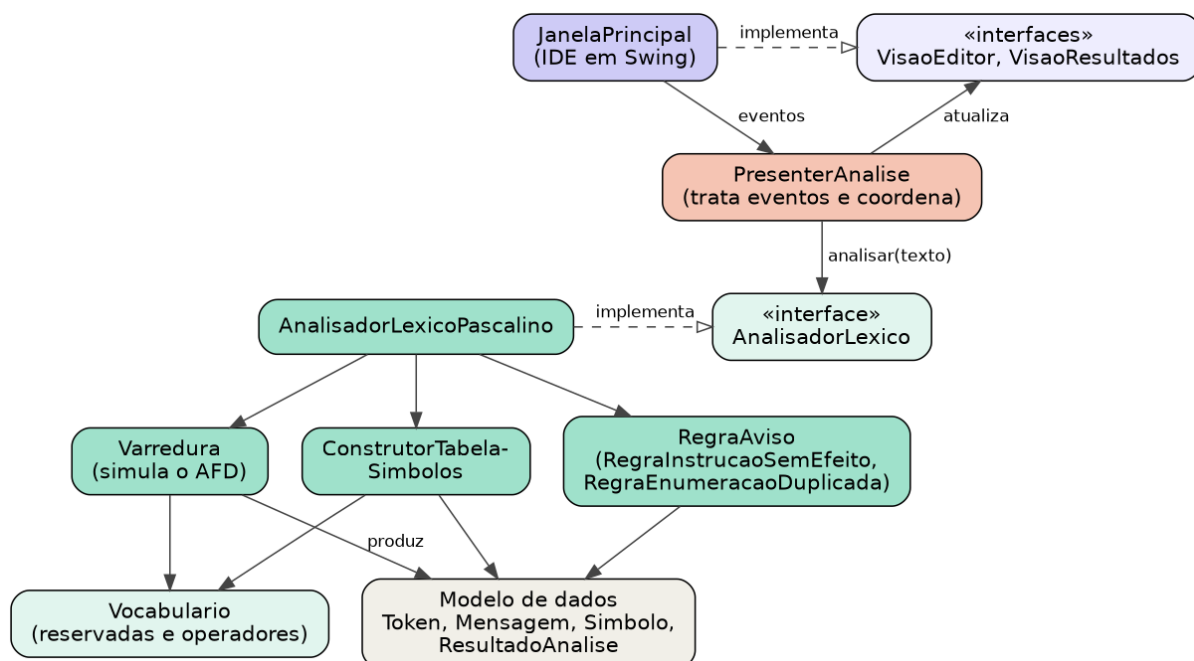


Figura 1 – Diagrama dos módulos do programa. As setas tracejadas indicam implementação de interface; a classe Aplicacao, responsável por criar e conectar os módulos, foi omitida.

A tabela a seguir resume a função de cada módulo.

Módulo	Responsabilidade
Aplicacao	Ponto de entrada da aplicação. Cria o analisador, a janela e o presenter e conecta esses componentes.
JanelaPrincipal, NumeracaoLinhas, PintorSublinhadoOndulado	Visão em Swing, com editor numerado, sublinhado ondulado para erros e avisos, barra de ferramentas e abas de resultado.
VisaoEditor, VisaoResultados	Interfaces que definem as operações disponíveis para o presenter na visão: ler e definir o texto, marcar problemas e exibir mensagens, tokens, símbolos e status.
PresenterAnalise	Recebe os eventos da janela — texto alterado, compilação, carregamento de exemplo e seleção de mensagem —, chama o analisador e encaminha o resultado para a visão.
AnalizadorLexico, AnalizadorLexicoPascalino	Interface e implementação do analisador, responsáveis pela varredura, pelas regras de aviso e pela tabela de símbolos.
AnalizadorLexicoPascalino	Implementação do analisador. Executa a varredura, aplica as regras de aviso e monta a tabela de símbolos.
Varredura	Interface e regras, organizadas com o padrão Strategy, que examinam os tokens e geram avisos.
Vocabulario	Reúne as constantes da linguagem, como palavras reservadas, operadores, limite de 15 caracteres e delimitadores de sincronização.
RegraAviso, RegraInstrucaoSemEfeito, RegraEnumeracaoDuplicada	Interface e regras, organizadas com o padrão Strategy, que examinam a lista de tokens e geram avisos (warnings).
ConstrutorTabelaSimbolos	Monta a tabela de símbolos a partir dos tokens conhecidos e dos identificadores encontrados no programa.
Token, Mensagem, Severidade, Simbolo, ResultadoAnalise	Classes de dados compartilhadas entre os módulos.

4. O IDE

A IDE (Figura 2) reúne um editor com numeração de linhas, uma barra de ferramentas e um painel inferior dividido em três abas.

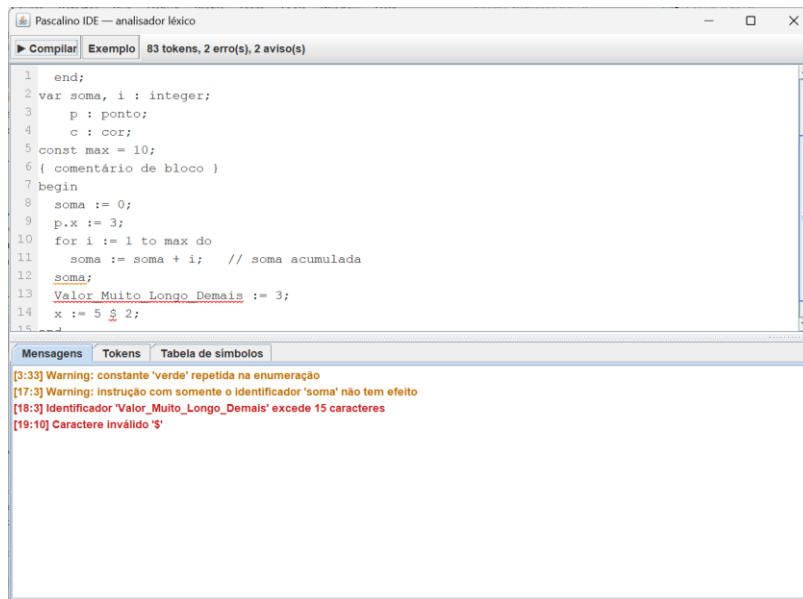


Figura 2 – Tela principal da IDE.

Os elementos da interface são:

- Editor. Utiliza fonte monoespaçada e destaca, na numeração, a linha em que o cursor está posicionado.
- Compilar e Exemplo. O botão Compilar executa a análise léxica e resume o resultado na barra de status. O botão Exemplo carrega um programa de demonstração que contém erros e avisos.
- Abas. A aba Mensagens exibe erros em vermelho e avisos em laranja, no formato [linha:coluna]; ao clicar em uma mensagem, o trecho correspondente é selecionado no editor. A aba Tokens mostra tipo, lexema, posição e atributo, enquanto a aba Tabela de símbolos apresenta os símbolos identificados.

Um `DocumentListener` acompanha as alterações no texto e reinicia um temporizador de 150 ms a cada tecla pressionada. Assim, a análise é executada quando o usuário faz uma pausa, evitando processamento desnecessário durante a digitação. Os erros aparecem sublinhados em vermelho, e os avisos, em laranja. A contagem de linhas e colunas começa em 1, e cada tabulação conta como uma coluna.

5. O analisador léxico

O `AnalizadorLexicoPascalino` trabalha em três etapas. Primeiro, a `Varredura` gera os tokens e registra os erros léxicos. Em seguida, cada `RegraAviso` percorre os tokens e acrescenta os avisos

identificados. Por fim, o ConstrutorTabelaSimbolos monta a tabela de símbolos. Ao final, as mensagens são ordenadas pela posição em que aparecem no código.

A Varredura simula o funcionamento do AFD. O método executar() observa o caractere atual e encaminha o processamento para o caso correspondente: espaço, comentário, identificador, número, literal, operador ou caractere inválido. Os operadores formados por dois caracteres (:=, <>, <=, >=) são reconhecidos antes dos operadores de um caractere, garantindo, por exemplo, que <= seja tratado como um único token.

- Identificadores e palavras reservadas. Lexemas com mais de 15 caracteres geram erro. Nos demais casos, o lexema é convertido para maiúsculas e comparado com a lista de palavras reservadas; quando não há correspondência, é gerado um token ID.
- Números. O ponto só é incorporado ao número quando o caractere seguinte é um dígito. Esse olhar para o próximo caractere evita que end. seja interpretado como um número real. Se uma letra ou um sublinhado vier logo depois do número, como em 12abc, o trecho é considerado um número malformado.
- Literais. São lidos até o apóstrofo de fechamento. Se esse apóstrofo não for encontrado antes do fim da linha, um erro é registrado.
- Comentários. São descartados e não geram tokens. Quando um comentário de bloco não possui delimitador de fechamento, o analisador registra um erro.

A Figura 3 apresenta o AFD utilizado pelo analisador. Os estados finais correspondem aos tokens descritos no Apêndice A. Já os comentários e os espaços em branco retornam ao estado inicial sem produzir tokens.

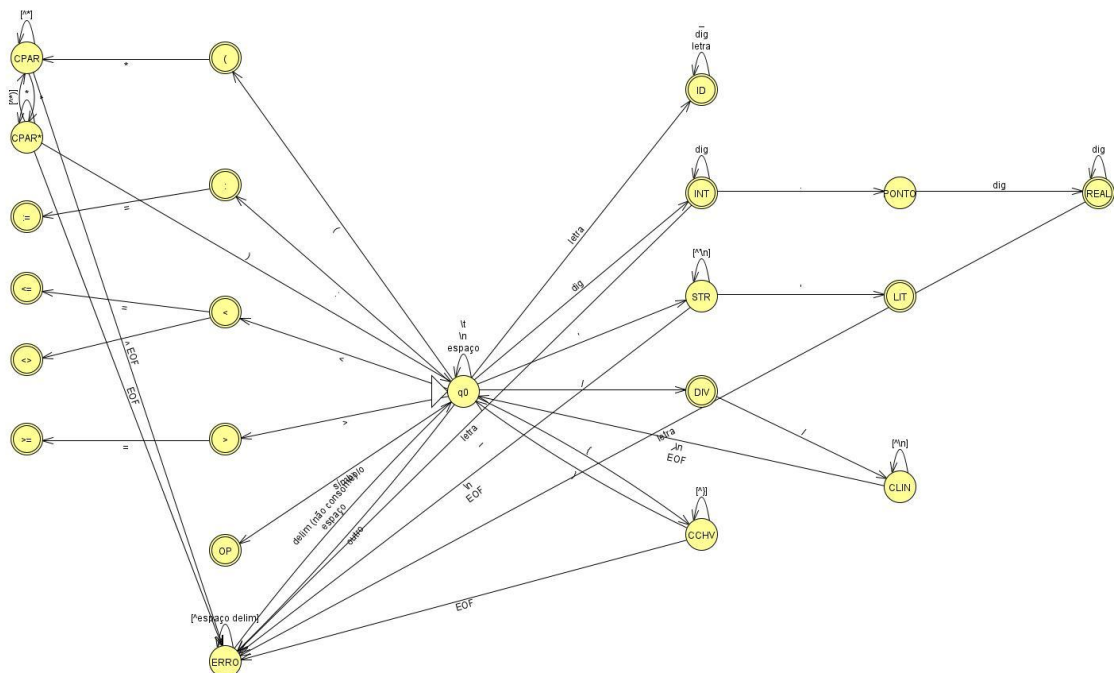


Figura 3 – Autômato finito determinístico do analisador léxico do Pascalino.

A tabela começa com os tokens conhecidos: 27 palavras reservadas, 4 operadores de dois caracteres e 15 símbolos de um caractere, totalizando 46 entradas. Depois, cada ID encontrado no programa é incluído uma única vez, acompanhado da linha de sua primeira ocorrência. Como os identificadores são armazenados em minúsculas, Soma e soma correspondem à mesma entrada. Cada registro contém o símbolo, a categoria e a primeira linha em que ele aparece; para os tokens pré-carregados, esse campo recebe o valor -. Identificadores com mais de 15 caracteres geram erro e não são adicionados.

Cada erro é registrado com sua posição, linha e coluna. Depois disso, a varredura continua, permitindo que vários problemas sejam encontrados em uma única execução.

Erro	Mensagem	Recuperação
Caractere inválido (por exemplo \$, @, #)	Caractere inválido 'c'	Modo pânico: descarta caracteres até um espaço em branco ou um delimitador de sincronização.
Identificador com mais de 15 caracteres	Identificador 'x' excede 15 caracteres	Consome o lexema inteiro, reporta o erro e segue sem gerar token.
Número seguido de letra ou sublinhado (12abc)	Número mal formado (identificador não pode começar com dígito)	Consome o restante do lexema (letras, dígitos e sublinhados).
Literal sem apóstrofo de fechamento	Literal não fechado	Descarta até o fim da linha.
Comentário de bloco sem fechamento	Comentário não fechado	Consome até o fim do arquivo; o sublinhado marca a abertura.

Modo pânico. Quando encontra um caractere que não pode iniciar nenhum token, o analisador descarta os caracteres seguintes até chegar a um ponto de sincronização — um espaço ou um dos delimitadores ; , () []. O trecho é então registrado como um único erro, reduzindo a ocorrência de mensagens em cascata.

Os avisos não interrompem a compilação. Cada regra implementa RegraAviso, seguindo o padrão Strategy, e examina a lista de tokens em busca de situações específicas:

- Instrução sem efeito. É identificado um ID que aparece isoladamente entre um ponto de início — ;, BEGIN, THEN, ELSE, DO ou REPEAT — e o próximo ;, como ocorre em soma;.
- Constante repetida em enumeração. É emitido um aviso quando o mesmo nome aparece mais de uma vez na declaração, como em (vermelho, verde, azul, verde).

6. Exemplo de execução

O botão Exemplo carrega o programa a seguir. Ele reúne uma enumeração, um registro, um laço for e comentários, além de conter três situações problemáticas:

```
program Demo;
type
  cor = (vermelho, verde, azul, verde);
  ponto = record
    x, y : integer;
  end;
var soma, i : integer;
```

```

    p : ponto;
    c : cor;
const max = 10;
{ comentário de bloco }
begin
    soma := 0;
    p.x := 3;
    for i := 1 to max do
        soma := soma + i;    // soma acumulada
    soma;
    Valor_Muito_Longo_Demais := 3;
    x := 5 $ 2;
end.

```

Os comentários são removidos e, por isso, não aparecem entre os tokens. Para esse exemplo, as mensagens esperadas são:

Posição	Tipo	Mensagem
3:33	Aviso	Warning: constante 'verde' repetida na enumeração
17:3	Aviso	Warning: instrução com somente o identificador 'soma' não tem efeito
18:3	Erro	Identificador 'Valor_Muito_Longo_Demais' excede 15 caracteres
19:10	Erro	Caractere inválido '\$'

Nesse caso, a barra de status informa 2 erros e 2 avisos, que aparecem sublinhados no editor, como mostrado na Figura 2.

7 Conclusão

Como resultado, foi desenvolvido um analisador léxico completo para o Pascalino, acompanhado de uma IDE que apresenta tokens, tabela de símbolos e problemas em tempo real. A solução atende aos requisitos do enunciado, incluindo o limite de 15 caracteres, a ausência de distinção entre maiúsculas e minúsculas, a remoção de comentários, a indicação de linha e coluna com sublinhado, a tabela de símbolos, o modo pânico, o AFD e a geração de avisos. As extensões implementadas também deixam a linguagem preparada para as próximas etapas do compilador.

Apêndice A – Tabela de tokens

As expressões regulares apresentadas a seguir não diferenciam letras maiúsculas de minúsculas. O atributo corresponde à informação armazenada no token; o símbolo - indica que não há atributo.

A.1 Classes com atributo

Token	Lexema	Expressão regular	Atributo	Valor
ID	soma, x1, valor_2	<code>[A-Za-z][A-Za-z0-9_]{0,14}</code>	lexema em minúsculas (chave da tabela de símbolos)	identificador de variável, constante, tipo, procedimento ou função
NUM	10, 3.14	<code>[0-9]+(\.[0-9]+)?</code>	texto do número	valor numérico (inteiro ou real)
LITERAL	'ola mundo'	<code>'[^'\n]*'</code>	conteúdo sem os apóstrofes	cadeia de caracteres

A.2 Palavras reservadas

Token	Lexema	Expressão regular	Atributo	Valor
PROGRAM, BEGIN, END, CONST, VAR, TYPE, RECORD	program, begin, end, const, var, type, record	o próprio lexema	-	estrutura do programa e das declarações
INTEGER, REAL, CHAR, STRING	integer, real, char, string	o próprio lexema	-	tipos básicos
PROCEDURE, FUNCTION	procedure, function	o próprio lexema	-	sub-rotinas
IF, THEN, ELSE	if, then, else	o próprio lexema	-	desvio condicional
WHILE, DO, REPEAT, UNTIL, FOR, TO, DOWNT, BREAK, CONTINUE	while, do, repeat, until, for, to, downto, break, continue	o próprio lexema	-	repetição e controle de laço
OU, E	ou, e	o próprio lexema	-	operadores lógicos

A.3 Operadores e pontuação

Token	Lexema	Expressão regular	Atributo	Valor
:=, <>, <=, >=, =, <, >	:= <> <= >= = < >	o próprio símbolo	-	atribuição e relacionais
+, -, *, /	+ - * /	o próprio símbolo	-	operadores aritméticos
() [] , ; : .	() [] , ; : .	o próprio símbolo	-	delimitadores, separadores, fim do programa e acesso a campo

A.4 Elementos descartados (não geram token)

Elemento	Expressão regular	Tratamento
Espaço em branco	[\t\r\n]+	ignorado
Comentário de linha	//[^\\n]*	removido
Comentário entre chaves	\{[^}]*\}	removido
Comentário entre parênteses e asterisco	\(*([^*] *+ [^*])) **+\\)	removido